scheduling

Techiche

Rappresentazione

diagramma di Gnatt
rettangoli affiancati da t0 a tn

caso multi-risorsa si fanno
più diagrammi paralleli

processo alterna
CPU|I/O burst o
bound se lunghi

Tipi

cause context-switch:
1. running->waiting
(I/O, SC bloccante)
2. running->ready(int)
3. waiting->ready
4. kill process,
1,4 sostituito
2,3 processo riprende

-S. cooperativo: 1,4
semplice,
+S. preemptive: 1-4
usa meglio risorse

Scelta scheduler

* %t cpu esegue processi max
* throughput
n.processixUnitaTempo
dipende len(processo), max
* tempo turnaround
tFine-tInizio processo, min
* t. attesa nella coda
ready, min
* t.1°Risposta-t.sottomissione
processo , min

Soluzioni semplici

FCFS: coda FIFO, no preemption
-t. attesa e turnaround
-ordine influenza

SJF: eseguito processo ready
con minimo cpu burst,
no preemption (rischio starvation)
+min tempo attesa
-inimplementabile, solo
approssimazioni grazie scheduler:
* long-term predice durata job
* short-term media job precedenti
** no preemptive, p. non interrotto
** si preemptive, p. interrompibile se
p2burst<p1burstRimanente
(SJF-time First)

timeSlice

t.max esecuzione
di un processo,
-(TSbreve->+ContSwitch)
-(TSlungo->Inattività p.visibile)

Implementazioni

coda FIFO p.ready:
rilascio cpu
* volontario (I/O)
* timeSlice esaurito,
aggiunto in coda se
CPUburst incompleto

timer HW (I/O) per CPU
scatena interrupt

Round-Robin

Priorità

base: stesse possibilità
a p. d'esecuzione

realtà: p. diversa
priorità,
scheduler sceglie
p. max priorità

priorità di OS o esterne
calcolate vari modi

priorità statica (-starvation:
p. accantonati
per p. priorità più alta)

priorità dinamica (+aging:
incremento priotià p.ready
fino priorità max)

classi

processi con caratteristiche
simili e priorità comune

sotto-code ready
(una per classe)

classi diverse politiche

sceduler multilivello:
seleziona p.ready secondo
politica classe da coda
non vuota max priorità

Real-Time

correttezza=valore+isanteOut
Hard/Soft real-time
processi periodici/aperiodici(evento)

tipi: Rate Monotonic, EDF

Scheduler

Cos'è

gestire avvicendamento
processi (ordine esecuzione)
e inizio/fine I/O

Terminologia
Schedule: sequenza temporale
Scheduling: azione calcolo
Scheduler: SW

OS

os gestisce risorse come
memoria, I/O,
file system, processi
tramite strutture dati (tabelle)

tabella processi PCB per processo

Azionamento

int SW/HW **mode
switching**: use→kernel

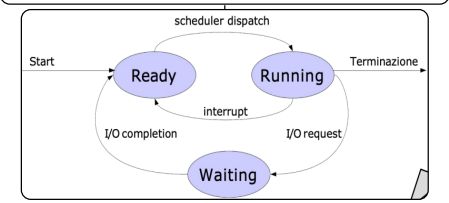
può avvenire
context switching:
1. p1 salva stato in PCB
2. recupero stato p2 da PCB
3. cambio

Processi

```
Allocazione:
* stack, funzioni (max) ↓
* heap, allocazioni dinamiche ↑
* dati
* codice sorgente (0)

PCB, Descrittore del processo:
Identificazione
   pid, ppid, userIdExecuter
Stato
   registri generali e specifici
   (PC, registri stato)
Controllo (per scheduling)
      stato (code di processi)
        running
        waiting, attesa evento per completare
          (finisce prima del quanto)
        ready, eseguibile ma attende CPU liberra
      info alg. scheduling: priorità, *coda
      PID di waiting
      idx min max e *tabellePagine
      accounting:
         t.esecuzione, somma quanti
         t.attivazione, attimo start
      risorse controllate
      info IPC

Gerarchia: padre/creatore→figlio/creato
+semplice, +ereditarietà,
figlio è processo o thread
```



Thread

Idea

Passato: processo ha
"linea di controllo"
1istruzione per volta

Oggi: processi
"multithreaded"
(+linee di controllo),
benefici:
* +lavoro di gruppo
* -uso memoria
* context switchig
economico

Cos'è

unità base uso CPU

thread stesso processo:
* propri: copia
stato processore, PC, stack
* condivisi: codice, dati, I/O
(processo lightweight)

Implementazioni

User thread (thread library)
efficente, -kernel single-threaded,
Kernel thread (os)
+cpu libera
-lento per mode switching

Many(user)-to-One(kernel)
One(user)-to-One(kernel)
-scalabilità
Many(user)-to-Many(kernel)